FR

Fee Reconciliation Desk

FEE PAYMENT MATCHING AND RECONCILIATION

TECHNICAL DESIGN

Technical Design

Architecture, data model, processing logic, financial controls and security

Document ID FRD-TD-05 · **Version** 1.0 · **Date** 27-Oct-2026

Prepared by Yasin Jariwala

Audience Developers, reviewers, Finance controls, IT and security teams

Document control

Document	Technical Design
Document ID	FRD-TD-05
Version	1.0
Date	27-Oct-2026
Prepared by	Yasin Jariwala
Product	Fee Reconciliation Desk v1.0
Audience	Developers, reviewers, Finance controls, IT and security teams
Classification	Public
Source	github.com/yasin679/FeeReconciliationApp · docs/technical-design.md

Version history

Version	Date	Author	Changes
1.0	27-Oct-2026	Yasin Jariwala	First release

Related documents

ID	Document
FRD-UG-01	User Guide
FRD-IG-02	Installation and Administration Guide
FRD-QS-03	Quick Start
FRD-FAQ-04	FAQ and Troubleshooting
FRD-UAT-06	UAT Test Cases
FRD-RN-07	Release Notes v1.0

Contents

1 Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Design principles 3
2 Architecture 5
2.1 Overview 5
2.2 Components 5
2.3 Technology choices 6
3 Data model 7
3.1 Txn (payment) 7
3.2 Proposal, Allocation, LedgerEntry, Receipt 7
3.3 Students, users, cases, policy and audit 8
4 Processing logic 9
4.1 Payment life cycle 9
4.2 Duplicate detection 9
4.3 Split remarks 9
4.4 Approval and posting 9
4.5 Reversals 10
4.6 Corrections 10
4.7 Concurrency 10
4.8 Books check 10
5 Roles and permissions 11
6 Security 12
7 API reference 13
8 Test results 14
9 Known limitations 15
10 Deployment 16

1 Introduction

1.1 Purpose

This document describes how Fee Reconciliation Desk is built. It covers the architecture, the data model, the rules applied at each gate, the security controls, the API and the test evidence. Its readers are engineers who will extend the system, reviewers who must sign it off, and Finance and security teams who need to know how the controls are enforced.

1.2 Scope

Version 1.0 covers the following:

- Bank statement import and duplicate detection
- Student matching
- Allocation with maker-checker approval
- Idempotent posting to a fee ledger
- Receipts
- Linked reversals and compensating corrections
- Cash reporting and verification
- Student claims and mismatch cases
- A hash-chained audit log
- Role-based access for six roles

It does not cover the following:

- Refund payouts
- A live bank API
- A live connection to the student-management system. Posting writes to the application's own ledger, behind the same interface a connector would use.

1.3 Design principles

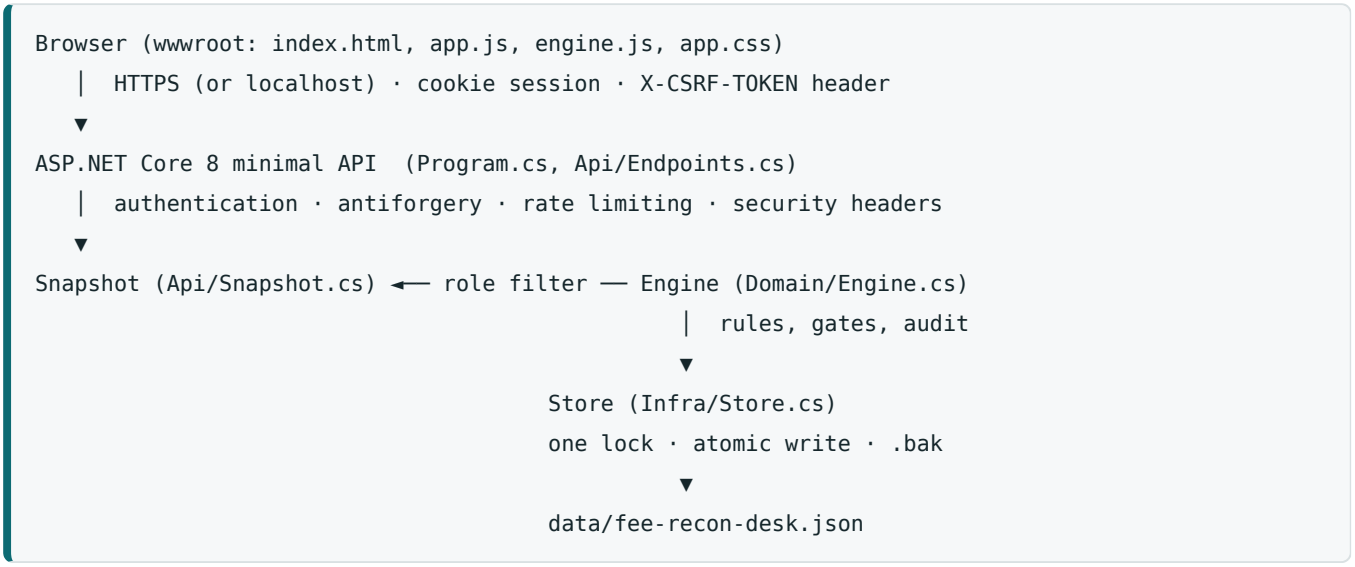
Principle	How it is applied
Separate the questions	Each payment carries separate states for verification, identification, allocation, posting and receipt, so "we know who paid" can never be confused with "the money is on the ledger"
Verify before allocating	Only bank-feed lines and Finance-verified cash can be allocated. Claims and cash reports never touch a balance.
Never guess	A name or phone match produces a candidate, never a match. The fee component is never inferred unless Finance switches on an order (AA-1).
Two people, by user	Proposer ≠ approver, correction requester ≠ approver, cash reporter ≠ verifier. Checked against the user ID, not the role.
Append, never edit	Reversals, corrections and receipt changes are new linked records. Nothing that touches money is deleted.
Server decides	The browser only displays. Every rule is enforced in the C# engine, and refusals are audited.

Principle	How it is applied
No dependencies	ASP.NET Core 8 only, with no NuGet packages. `nuget.config` clears all sources, so it builds offline.

2 Architecture

2.1 Overview

The application is a single ASP.NET Core 8 process. It serves the static screens and a JSON API. The API calls a rules engine, which works on one in-memory state object. A store serialises every request through one lock and saves the state to a JSON file after each change.



2.2 Components

Component	File	Lines	Responsibility
Start-up and pipeline	`Program.cs`	89	Configuration, data protection keys, cookie authentication, antiforgery, rate limits, security headers, `--selftest`
Endpoints	`Api/Endpoints.cs`	266	Sign-in, state, actions, import, export, users, audit verification
Snapshot	`Api/Snapshot.cs`	74	Builds the state sent to the browser, filtered by role
CSV	`Api/Csv.cs`	79	Bank statement parsing and reconciliation export
Engine	`Domain/Engine.cs`	769	All business rules: matching, allocation, approval, posting, reversals, corrections, cash, claims, cases, policy
Data model	`Domain/Models.cs`	277	The state classes listed in section 3
Sample data	`Domain/Demo.cs`	62	Students S101–S104 and payments P1–P9
Self-tests	`Domain/SelfTests.cs`	244	The 27 scenarios TC-01 to TC-27
Store	`Infra/Store.cs`	71	Locking, loading, atomic save, rolling backup
Security	`Infra/Security.cs`	65	PBKDF2 password hashing, audit hash chain
Bootstrap	`Infra/Bootstrap.cs`	69	First start: admin password, demo accounts
Screens	`wwwroot/*`	2,104	Rendering, editor validation (repeated on the server), role-aware buttons

2.3 Technology choices

Choice	Reason
ASP.NET Core 8 minimal API	Long-term support, runs on Windows, Linux and macOS, small surface
No NuGet packages	Nothing to patch or audit beyond the framework; offline build
JSON file store with a single lock	Enough for one college's volumes and no database to install. The lock makes every action all-or-nothing and removes race conditions.
Cookie sessions with antiforgery	Same-origin browser app; no tokens to keep in browser storage
Plain JavaScript front end	No build step; the same `engine.js` powers the offline preview

3 Data model

All amounts are whole rupees stored as `long`. All times are `DateTimeOffset`.

3.1 Txn (payment)

Field	Meaning
`Id`	Internal ID, for example `P6`
`Source`, `SourceTxnId`	`bank`, `cash` or `upload`, and the bank's UTR or reference
`Kind`	`payment`, `reversal`, `cash` or `claim`
`Amount`, `Channel`, `PayerName`, `PayerContact`, `Remark`	As imported
`ReceivedAt`, `IngestedAt`, `Batch`	When the money moved, when it was imported, and from which file
`Verification`	`verified`, `pending`, `not_found`, `linked` or `duplicate`
`DuplicateOf`, `IdConflict`	The earlier record this repeats; set when the same ID arrives with a different amount
`Candidates`, `Matched`, `MatchBasis`	Candidate students with confidence and reason; the matched student and why
`Split`	Lines parsed from a split remark
`ReversalOfRef`, `Reversal`, `ReversedApplied`	For reversals: the original, the link details and the amount applied
`ClaimedUtr`, `LinkedClaims`, `LinkedBankTxn`	Links between a student claim and the bank line
`CashEvidence`, `ReportedBy`	Receipt-book number, deposit slip and who reported the cash
`Hold`, `Notes`, `Flags`	Evidence requests, escalations and warnings
`Version`	Incremented on every change; used to reject stale edits

3.2 Proposal, Allocation, LedgerEntry, Receipt

Entity	Key fields	States
Proposal	`Id`, `TxnId`, `Proposer`, `Reason`, `Evidence`, `AllocIds`, `Approver`, `Note`	`proposed` → `approved` → `posted`, or `post_failed` → retry, or `rejected`
Allocation	`Id`, `TxnId`, `PropId`, `Sid`, `Comp`, `Amount`, `Reversed`, `Overpay`, `CorrReq`	`proposed`, `posted`, `rejected`, `corrected`
LedgerEntry	`Id`, `Key` (unique), `Sid`, `Comp`, `Amount`, `Type`, `TxnId`, `OrigTxnId`, `AllocId`	`payment`, `reversal`, `correction`
Receipt	`No`, `AllocId`, `Sid`, `TxnId`, `Comp`, `Amount`, `Adjustments`, `CancelReason`	`issued`, `adjusted`, `cancelled`

3.3 Students, users, cases, policy and audit

Entity	Key fields
Student	ID, name, registered contact, guardian, fee obligations by component
User	`UserName`, `DisplayName`, `Role`, `StudentId`, `Enabled`, `PasswordHash`, `Salt`, `Iterations`, lockout counters, session stamp
MismatchCase	`Id`, `Sid`, `Ref`, `Text`, `Status`, `Resolution`, `ResolvedBy`
Policy	`AutoMatch` (AM-1), `ComponentPolicy` (AA-1), `MaxOpenClaims`, `SmsFail` (demo), fee amounts
AuditEvent	`Id`, `Ts`, `Actor`, `Person`, `Role`, `Action`, `Entity`, `Txn`, `Detail`, `Result` (`ok`, `denied`, `blocked`, `failed`), `PrevHash`, `Hash`

4 Processing logic

4.1 Payment life cycle

Gate	Rule in the engine	Evidence in the audit log
1 Verified	Bank-feed lines are verified on import. Cash moves from `pending` to `verified` only through `VerifyCash` by Finance, never by the reporter. Claims stay `not_found` until a bank line with the same normalised UTR arrives.	Import, cash verification, claim link
2 Matched	AM-1: exactly one valid, registered student ID in the remark and no conflicting signal. Otherwise candidates from name or contact, graded High, Medium or Low.	Auto-match or candidate list
3 Allocated	Lines total ≤ received − reversed − already proposed or posted. Every line needs student, component and a whole-rupee amount. A name-only student needs recorded evidence.	Propose allocation, with lines, reason and evidence
4 Posted	A Finance approver who is not the proposer. Free funds are re-checked at approval. Posting uses the proposal ID as idempotency key.	Approve, post or post failed
5 Receipted	One receipt per posted allocation line	Receipt issued

4.2 Duplicate detection

The transaction ID is normalised by upper-casing it and removing spaces and punctuation. The pair (source, normalised ID) must be unique.

- **Same ID, same amount:** the line is stored as `duplicate`, linked to the original, and never counted.
- **Same ID, different amount:** the line is flagged `IdConflict`. It is never counted until Finance closes it with a note.

A claim's UTR is matched by the same normalisation, so `430219884120` links to `UPI/430219884120`. If two students claim one payment, both are downgraded and flagged.

4.3 Split remarks

A remark such as `S101 15000 / S104 15000` is parsed into split lines. The lines are used only when every student ID is valid and the amounts add up to the payment. They pre-fill the allocation editor and are proposed and approved as one proposal, so a split is never half-posted.

4.4 Approval and posting

1. `Approve` checks the role (Finance approver), that the approver is not the proposer, and that the free funds still cover the proposal. Any line above the outstanding fee needs an approval note.
2. Posting writes one `LedgerEntry` per allocation. Each entry is keyed by the allocation ID, so it can only exist once.
3. If the student system fails, the proposal becomes `post_failed` and nothing is written. `Retry` repeats the post with the same idempotency key. A proposal already `posted` returns a no-op.

4. One receipt is issued per line.

4.5 Reversals

A reversal must have a negative amount and must name a real, non-duplicate bank payment. The amount is capped at what is still unreversed. When a Finance approver applies it, a negative ledger entry is written against each affected allocation, keyed by reversal and allocation so it cannot be applied twice. The receipt gets an adjustment record and the original payment keeps its full amount. Anything outside these rules becomes a *Reversal exception*, to be re-linked or closed with a note.

4.6 Corrections

A correction is requested by one person and approved by a different Finance approver. It writes a negative `correction` ledger entry for the net posted amount and cancels the receipt with the reason. The allocation is marked corrected, which returns the money to unallocated. No earlier record changes.

4.7 Concurrency

All requests pass through one lock in the store. Each payment carries a `Version`. The editor sends the version it loaded, and a proposal against an older version is refused with *This payment changed while you were editing*. In testing, 20 simultaneous proposals and parallel approvals on one ₹25,000 payment produced exactly one proposal, one approval, one posting and one receipt.

4.8 Books check

At any moment: verified net money = posted + awaiting approval + unallocated. The difference is calculated on every snapshot, shown on the Overview and written at the foot of the reconciliation export. It stayed at ₹0 throughout the automated tests and the random-action test.

5 Roles and permissions

Permission	Ops	Counsellor	Finance reviewer	Finance approver	Admin
View queue	✓	✓	✓	✓	✓
Propose allocation	✓		✓	✓	
Approve allocation				✓	
Retry posting	✓		✓	✓	
Report cash	✓	✓			
Verify cash			✓	✓	
Apply reversal				✓	
Request correction	✓		✓	✓	
Approve correction				✓	
Request evidence	✓	✓	✓	✓	
Escalate	✓	✓	✓		
Resolve cases	✓		✓	✓	
Import bank statement			✓	✓	
View audit, export			✓	✓	✓
System check			✓	✓	✓
Policy, users, students					✓

Students can submit claims and raise cases for their own student ID only. On top of this table, the two-person rules in section 1.3 are checked against the user ID.

6 Security

Area	Control
Passwords	Salted PBKDF2-SHA256 with 210,000 iterations and constant-time comparison. At least 10 characters with three character classes.
Sign-in	Same message for an unknown user and a wrong password. A dummy hash is computed for unknown users so timing does not reveal user names. Lockout for 10 minutes after 5 failures. Rate limit per address (`LoginAttemptsPerMinute`).
Sessions	Cookie authentication: HttpOnly, SameSite=Strict, idle timeout. A session stamp on each user ends sessions on disable, role change or password reset.
Request forgery	Antiforgery token in the `X-CSRF-TOKEN` header on every change
Rate limiting	Sign-in per address; API 600 requests per minute per user
Headers	`Content-Security-Policy` (self only, no frames), `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, a restrictive `Permissions-Policy`. No server header.
Data exposure	Snapshots are filtered by role. Students receive only their own student record, ledger, receipts, claims and cases. A claim never reveals whether a matching bank line exists.
Input	Request bodies limited to 6 MB, CSV to 5 MB and 20,000 rows. Text fields length-limited, amounts range-checked (up to ₹10 crore), at most 20 lines per proposal. Malformed JSON is treated as empty.
Output	CSV export neutralises cells starting with `=`, `+`, `-` or `@`. All text is HTML-escaped when shown.
Audit	Every action and refusal is recorded. Each entry's SHA-256 hash covers its fields and the previous hash, so any edit or deletion breaks the chain.
Storage	One lock; a temporary file plus atomic replace; rolling `.bak`. An unreadable file stops start-up rather than starting an empty book. Session keys are protected with DPAPI on Windows.
Hosts	`AllowedHosts` limits the host names served. Forwarded headers are honoured only when `BehindProxy` is set.

7 API reference

All routes are under `/api` except static files. Every POST needs the session cookie and the `X-CSRF-TOKEN` header.

Method and route	Who	Purpose
`GET /api/csrf`	Anyone	Antiforgery token
`POST /api/login`	Anyone	Sign in with user name and password
`POST /api/logout`	Signed in	Sign out
`GET /api/state`	Signed in	Role-filtered snapshot
`POST /api/act/{name}`	By permission	One business action (list below)
`POST /api/import/bank`	Finance	Bank statement CSV
`GET /api/export/reconciliation.csv`	Finance, admin	Reconciliation export
`GET /api/selftest`	Finance, admin	Run the 27 scenarios
`GET /api/audit/verify`	Finance, admin	Verify the audit hash chain
`GET /api/users`, `POST /api/users/{op}`	Admin	List, add, change role, disable, enable, reset password
`POST /api/admin/reset-demo`	Admin, sample data only	Restore the sample data

Actions for `/api/act/{name}`:

- **Allocation:** `propose`, `approve`, `reject`, `retry`
- **Reversals and conflicts:** `applyReversal`, `closeReversal`, `relinkReversal`, `closeConflict`
- **Cash:** `reportCash`, `recordCashEvidence`, `verifyCash`
- **Corrections:** `requestCorrection`, `correct`, `rejectCorrection`
- **Investigation:** `requestEvidence`, `escalate`, `recheckClaim`, `retryLookup`
- **Student requests:** `submitClaim`, `raiseCase`, `resolveCase`
- **Administration:** `setPolicy`, `registerStudent`

Each returns `{ ok, error?, data? }`.

8 Test results

Suite	Scope	Result
Built-in system check (C#)	TC-01 to TC-27, mapped to acceptance criteria AC-01 to AC-16	27 / 27 pass
Browser end-to-end	14 journeys with real sign-ins for every role, including import, approvals, reversal, correction, cash, user admin and persistence	14 / 14 pass, 0 page errors
API security	19 checks covering anonymous access, traversal, forgery, cookie flags, student isolation, 11 role-escalation attempts, two-person rule, session revocation, lockout, timing, malformed input, script and CSV injection, headers, host and method tampering, audit chain and secrets at rest	0 findings open
Concurrency	20 parallel proposals and approvals on one payment	1 posting, 1 receipt
Random actions (`tests/fuzz.js`)	24,000 random actions by every role across 60 runs on the rules engine, with and without posting failures and AA-1	No invariant violations; control difference ₹0 throughout

9 Known limitations

- **Single instance:** a single instance and a single data file, sized for one college.
- **No live integrations:** bank and student-system integrations are by CSV and the internal ledger.
- **Policy approval:** a policy change needs only one administrator.
- **Receipts:** receipts are on screen only; there is no PDF.
- **Fee amounts:** each component has one fixed fee amount, not an amount per student.

10 Deployment

The application can run on a laptop (`start.bat`), on a Windows server under IIS, on Azure App Service, or behind a reverse proxy on Linux. Production use needs the following:

- HTTPS
- `BehindProxy` and `AllowedHosts` set for the host
- `EnableDemoData` set to false
- Personal accounts for every user, with at least two Finance approvers
- A daily backup of the data folder

Details are in FRD-IG-02.